DevFest GDGoC Japan 2026 Hackathon

Submission Document

Team Information

Venue	Nagoya University (Nagoya)
Team Name	Contour
Member	David Belisario Patrick Santos, Le Minh Quang, Bert Jacob Andal Tropicales, Soo Wing Seng
Application Name	"SEAR" (Student Evacuation AR)
Category	Creative
GitHub Repository URL	`https://github.com/Devfest-Hackathon-2026/df-2026-t113.git`

1. Project Overview

"SEAR" (Student Evacuation AR) is a phone app that gets you where you need to go, fast, in your own language. In an emergency it finds the nearest official evacuation site, guides you to the nearest exit of the building, and points the way with 3D AR arrows on the camera view. On an ordinary day it is a campus guide that takes students to buildings and classrooms, floor by floor. A Gemini assistant answers questions about sites, warnings and directions.

- **Supported languages (expandable):** English, Japanese, Chinese and Korean
- **Platforms:** Android and iOS apps. Both share one interface, and native code adds the camera, sensor and AR features. A browser version is also available for quick testing, without the native features.
- **Data:** 1,735 official evacuation sites for Nagoya City (GSI open data) and Japan Meteorological Agency (JMA) warnings, refreshed automatically
- **Reliability:** Only official designations are shown. Opening status and capacity stay "unknown" instead of being guessed, and the assistant points users to official instructions.

2. Problems Addressed

2.1. Target Users

- University students who need to find a building or classroom fast
- Anyone who needs to reach the nearest evacuation site fast

2.2. Problems and Rationale

Problem: Whether an earthquake hits mid-lecture or a new student is racing to class, people lose precious minutes because alerts and evacuation data come in Japanese and nothing shows which exit, which way, or which classroom to head for.

Solution: SEAR gets you there fast, in your own language.

- **To the evacuation site, fast.** One tap shows the nearest official site. SEAR guides you to the nearest exit of the building, then 3D AR arrows on the camera view point the way. When an official alert arrives on Android, SEAR opens by itself with what to do now and the nearest suitable site.
- **To the classroom, fast.** Pick the building, your starting point and your destination, floor by floor, and follow the route on the floor map or with 3D AR.

2.3. Differences from Existing Approaches

Aspect	Before: official hazard maps and general map apps	With SEAR
Finding a site	Search a map service or data file, mostly in Japanese.	Nearest official sites on a map, filtered by disaster type, with distance and direction.
Getting there	Read a 2D map.	3D AR arrows on the camera view, a walking-route preview, and indoor directions to the nearest exit (demo).
Emergency alerts	The phone's alert text, in Japanese.	The app opens with the alert, what to do now and the nearest suitable site, in the app language.
Questions	None.	Gemini assistant that answers from the real site and warning data.
Languages	Mostly Japanese.	English, Japanese, Chinese and Korean, expandable to more, with offline translation.

3. Use of Google Technologies

3.1. Google Technologies Used (select at least one)

☐ App Engine
☐ Compute Engine
☐ Google Kubernetes Engine (GKE)
☒ Gemini (Vertex AI)
☒ Cloud Scheduler
☒ Google Maps JavaScript API
☒ ARCore
☒ Firebase App Distribution
☒ Cloud Run
☐ Firebase App Hosting
☒ Firebase Hosting
☒ Cloud Firestore
☒ Cloud Workflows
☒ ML Kit Translation
☒ Cloud Build

3.2. Examples of Google Technology Use

Technology	Use	Reasons / Benefits	Drawbacks
Cloud Run	Backend API, app interface and data-import jobs.	One container, scales to zero when idle.	Cold starts can delay the first request.
Firebase Hosting	Public HTTPS address in front of Cloud Run.	One address for Android, iOS and browsers.	One more layer to configure.
Gemini (Vertex AI)	SEAR AI assistant with function calling. Reads library sign photos.	Replies in four languages, grounded in our data.	Needs rate limits and strict rules so it does not invent sites.
Cloud Firestore	Stores evacuation sites and JMA warnings.	Serverless. Cached, so most requests cost no reads.	Charged per read and write.
Cloud Scheduler + Workflows	Import sites hourly and warnings every minute.	Retries and step graphs with no server to run.	Console edits can drift from the repository.
Google Maps JavaScript API	Basemap for sites and campus buildings.	Detailed, familiar map.	Needs the internet. Paid beyond the free tier.
ARCore	Library exit AR with depth-based obstacle checks (Apple ARKit on iPhone).	Uses the phone camera, no extra hardware.	Supported phones only. Depth detection is incomplete.
ML Kit Translation + speech input	Offline translation and voice input.	Works without the internet.	Language models download once first.
Cloud Build + App Distribution	Build the Android app on each push and send it to testers.	One-tap updates for teammates.	iOS builds still need Apple signing.

3.3. Why We Chose Google Technologies

We needed a working, multilingual app within a short hackathon. Google Cloud and Firebase let us put the website, the API, the database, the scheduled data imports and the AI assistant in one project, without managing servers.
Gemini's function calling was the key piece. Instead of letting the model answer from memory, the assistant calls our own tools to read the official site list and the latest JMA warnings, so its answers stay tied to real data.

4. Architecture Diagram

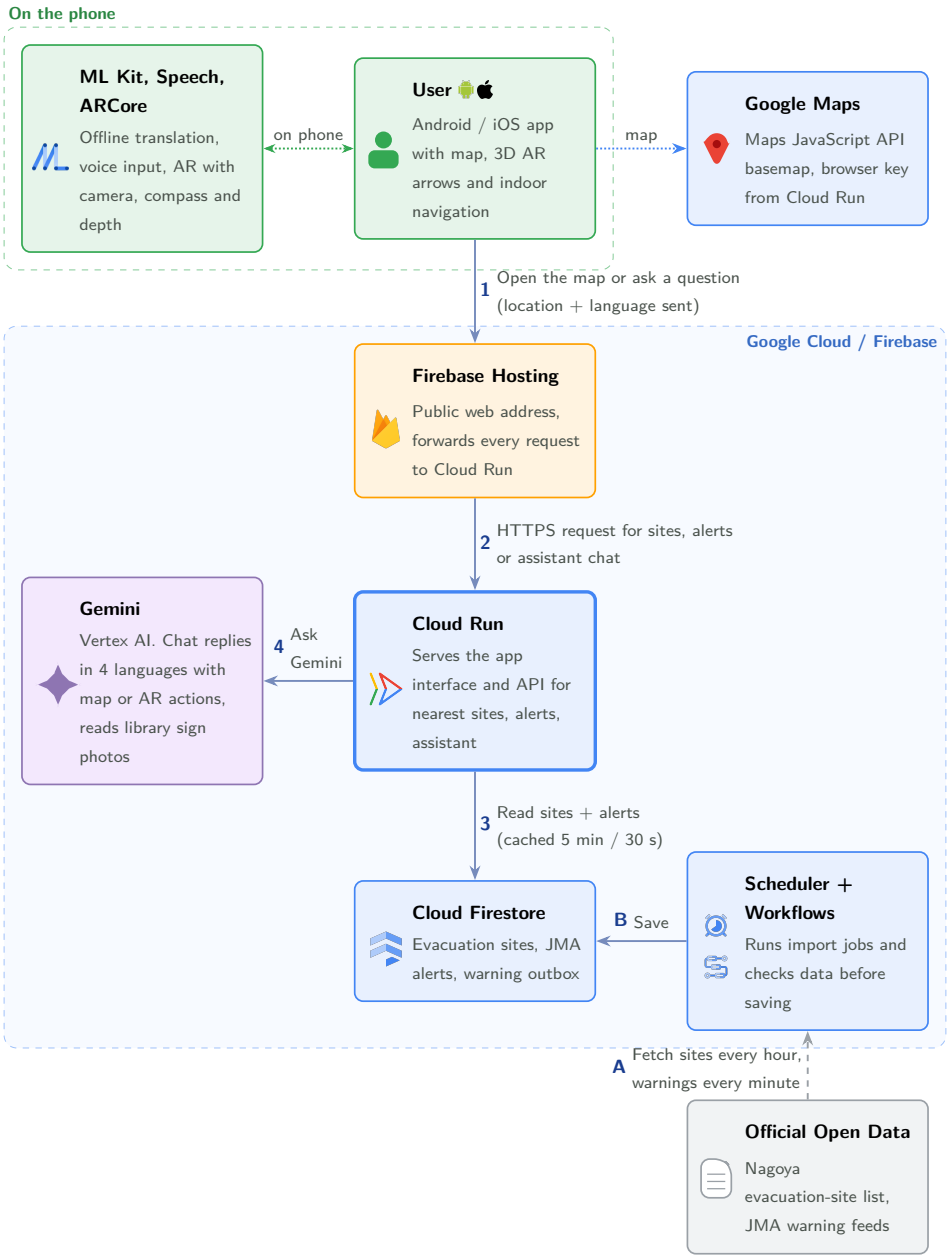


Figure 1: System architecture of SEAR. Steps 1–4 follow one user request from top to bottom, results return the same way. Steps A–B run on a schedule (evacuation sites hourly, JMA warnings every minute).

5. Deployed Project URL

Application URL: `https://gen-lang-client-0467526025.web.app/app/`

This is the browser version of the app (Firebase Hosting in front of Cloud Run). It opens on a phone or computer for quick testing. The Android and iOS apps have the same interface plus the native camera, sensor and AR features.

Installing the apps: Android testers install the app through Firebase App Distribution (by invitation). The iOS app is built and tested in the simulator but not yet distributed.

6. How to Test

6.1. Test Account Information

Item	Details
URL	`https://gen-lang-client-0467526025.web.app/app/`
Account	Not needed. The app has no login.

6.2. Test Procedure

Open the URL above, ideally on a phone. Choose a language (English, Japanese, Chinese or Korean) with the globe button in the header, or under **Menu** → **Settings**.

Student version

1. Tap **Student** at the top. Nagoya University buildings appear as blue pins with cards at the bottom.
2. On the **Central Library** card, tap **Details**, then **Indoor navigation**.
3. Choose the **floor**, your **starting checkpoint** and a **destination** (the 2F exit or the nearest stairs on that floor), then tap **I am at this checkpoint · start**. The route appears on the official floor map.
4. Back on the card, tap **3D AR directions**. In the Android app the camera opens straight to arrows towards the 2F Entrance Hall exit; tap the **Floor guess** or **Start guess** chip to correct it. Other buildings (no floor map yet) show one floor and one exit choice.
5. Drag the grey bar on top of a card down to shrink the cards to one row (name and **3D AR**), and drag it up to bring the full cards back.

Evacuation version

1. Tap **Evacuation** at the top. Official Nagoya evacuation sites appear as green pins, nearest first.
2. On the nearest site's card, tap **3D AR directions**. When asked "Are you inside a building right now?", tap **Yes, I'm inside**.
3. Choose your **floor** and tap **Start**. The exit of the nearest building is assigned (Central Library: 2F Entrance Hall; others: default main entrance) and 3D AR arrows lead to it. Follow exit signs and staff.

4. Once outside, tap **I'm outside · continue**. SEAR continues with **3D AR** arrows to the evacuation site.
5. (Android app only) Open the menu and tap **Test alert**. After a few seconds the emergency pop-up should open with what to do now and the nearest site.

6.3. Sample Data

No account or saved data is needed. The app uses:

- 1,735 designated emergency evacuation sites for Nagoya City (GSI open data, imported automatically every hour)
- Live JMA weather, earthquake, tsunami and volcano bulletins (checked every minute)
- 16 Nagoya University buildings for Student mode, and a Room 221 indoor demo in Engineering Building II

7. Additional Application and Feature Details

7.1. Main Features

#	Feature	Description
1	Nearest evacuation sites	Official Nagoya sites on a map, filtered by disaster type (earthquake, tsunami, flood, landslide, fire and more), with straight-line distance and direction.
2	3D AR directions	Arrows drawn on the camera view using the phone's compass point toward the chosen site.
3	SEAR AI (Gemini)	Questions in four languages. The assistant reads the real site list and warnings, and can highlight sites on the map or start AR.
4	Emergency pop-up	When an official emergency alert arrives, SEAR opens over the lock screen with what to do now and the nearest suitable site.
5	Indoor navigation	Guides to the nearest exit (avoiding elevators) or to a classroom, estimating position from steps, compass and air pressure. The Central Library uses the official university floor maps.
6	Student mode	Campus guide for 16 Nagoya University buildings that works offline.
7	Offline translation and voice input	ML Kit translates site names and text on the phone, even offline, and questions can be spoken.

7.2. Future Plans

- Push JMA warnings to phones with Firebase Cloud Messaging (the outbox already exists)
- Real floor plans collected and provided by the university, with more accurate indoor positioning, for more campus buildings
- Verified walking routes that take hazards and closures into account

- Distribute the iOS app to testers

8. Where, How and Why We Used AI

We used AI in two places: to build the app, and inside the app itself.

- **Development:** We wrote much of the code with AI coding agents. The repository's `AGENTS.md` sets the rules they follow: keep the code readable for a small team, never present demo data as real evacuation guidance, run the type checks, lint, tests and build, and report each change on our shared GitHub issue. This let a small team build the Android app, the iOS app, their shared interface and the backend within the hackathon.
- **SEAR AI (Gemini on Vertex AI):** The assistant uses function calling. It looks up the official site list and the latest JMA warnings with our own tools, then replies in the user's language and can show sites on the map or start AR. Grounding the answers in our data keeps them from inventing sites or warnings.
- **Photo landmarks (Gemini image understanding):** Inside the Central Library, a user can take a photo of a nearby sign. Gemini reads the sign text and suggests matching landmarks from our catalog. The user confirms the landmark before the position changes, because a model's guess should not move someone's location.
- **On-device AI:** On Android, ML Kit translates text offline and Android speech recognition turns spoken questions into text. On iOS, Apple Vision text recognition reads room labels on imported floor plans without uploading the image.

AI-generated code and answers were reviewed by development team members through functional testing and checks against the official data before being adopted. Code changes were merged through pull requests after automated checks (TypeScript type checks, lint, backend unit tests and browser UI tests). In the app, Gemini answers only from our tool results, requests are rate-limited, and replies point users to official instructions instead of guessing.

9. What We Gained and How Much We Grew Through Development

- **From open data to a usable service:** We learned to turn official files into something people can act on. Evacuation sites are imported every hour and JMA warnings every minute with Cloud Scheduler and Workflows, and checked before saving.
- **Keeping AI grounded:** Function calling taught us to let Gemini work from our data instead of its memory. Caching and rate limits taught us to watch cost from the start.
- **Limits:** Sensors alone cannot guide someone indoors reliably. Phone compasses drift and step counting adds up small errors, so good indoor navigation still needs human-informed data, such as accurate floor plans, checked routes and confirmed landmarks. Until that data exists for a building, we label routes as estimates.
- **Working as a team across platforms:** Sharing one interface and shared Kotlin code let Android and iOS move together. Pull requests and a shared GitHub issue kept four people working on the same app in a short time.